

# AI INTERVIEW SIMULATOR

## Project Defense, Architecture, UI Walkthrough & Viva Guide

**Candidate:** Srinivas Kandagatla | **Batch:** PFS-HYD-063 | **Stack:** React 18, Vite, TypeScript, FastAPI, PostgreSQL (Neon), Google Gemini 2.5 Flash | **Version:** 2.0 Live

### 1. Project Problem Statement

Technical interviews at modern tech firms assess three critical capabilities simultaneously: **conceptual clarity, live problem solving (DSA/system design), and verbal communication under pressure**. However, aspiring software engineers and job seekers face major hurdles during preparation:

- **Passive Memorization vs. Live Pressure:** Candidates study LeetCode and theory questions passively by reading or typing in isolation, but freeze when asked to verbally explain trade-offs and complexity to a live interviewer.
- **Prohibitive Cost of Human Mock Interviews:** Professional 1-on-1 mock interviews cost between \$100 to \$300 per session on platforms like Interviewing.io or Pramp, making consistent daily practice inaccessible for most students and job seekers.
- **Lack of Instant, Granular Feedback:** Self-practice provides no objective scoring on filler word frequency, technical accuracy, communication clarity, or omitted edge cases.
- **Fragmented Tools:** Candidates juggle separate tools for resume review, coding practice, behavioral questions, and mock interviews, with no unified system that links their resume projects directly to interview questions.
- **Anti-Cheating Reality:** Real corporate hiring rounds enforce full-screen proctoring and tab-switch penalties. Traditional practice platforms do not replicate this psychological pressure.

### 2. Our Solution & Complete Platform Features

The **AI Interview Simulator** solves these challenges by providing an enterprise-grade, browser-based interactive interview simulator powered by modern generative AI and Web APIs. Key capabilities include:

| Core Module                         | Key Features & Technical Implementation                                                                                                                                                                                                                 |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Voice-to-Voice AI Interview Cockpit | Full-duplex conversation. AI speaks questions aloud using Web Speech Synthesis (TTS). Candidate answers verbally using continuous Web Speech Recognition (STT). WebRTC webcam diagnostics and automated hardware track cleanup upon session completion. |
| Adaptive Generative AI Engine       | Powered by Google Gemini 2.5 Flash with strict JSON mode. Question #1 requests candidate intro; Question #2 dives into resume projects; subsequent questions dynamically probe technical gaps identified from previous answers.                         |
| Competitive Coding Arena            | 390 authentic LeetCode and GeeksForGeeks problems mapped across 20+ algorithmic patterns. Features VS Code Monaco Editor, Python 3 & JavaScript sandbox execution, input argument unpacking, and automated test-case evaluation.                        |
| AI Resume ATS Analyzer              | Instant PDF parsing extracting skills, projects, and work history. Calculates an ATS compatibility score (0-100) and automatically injects candidate project details into the interview engine for customized questioning.                              |
| Self-Paced Practice Question Bank   | Hundreds of categorized interview questions (DSA, System Design, SQL, OOP, Web, HR) with instant AI evaluation: scoring (1-10), strengths, weaknesses, and model 10/10 answers.                                                                         |
| Strict Anti-Cheating Proctoring     | Enforces browser full-screen mode, tracks document visibility loss (tab-switches) with a 3-warning auto-termination rule, and monitors 90-second candidate inactivity with an automated countdown.                                                      |
| Candidate Scorecard & PDF Export    | Radar charts for technical, communication, relevance, and completeness scores. Filler word frequency tracking, mistake timeline, and instant downloadable PDF report built with ReportLab.                                                              |

### 3. How to Explain Working in UI (Viva / Demo Walkthrough)

When demonstrating or explaining the platform to an interviewer, examiner, or evaluator, follow this structured, 8-step chronological flow:

- **Step 1: Landing Page & Platform Navigation:** Show the modern responsive landing page. Highlight the clean navigation bar linking to Interview, Coding Arena, Practice Bank, Resume Analyzer, and Analytics. Explain that the UI uses Tailwind CSS with dark mode aesthetics, Lucide icons, and responsive breakpoints.
- **Step 2: User Authentication (Register & Login):** Explain that candidates register with email, name, and password. Upon login, the backend issues a cryptographically signed JWT bearer token stored securely in localStorage. AuthContext in React manages global auth state and automatically redirects unauthenticated users.

- **Step 3: Resume Upload & Instant ATS Analysis:** Navigate to Resume Analyzer. Upload a PDF resume. Explain that the backend parses raw text and feeds it to Gemini AI. The UI displays an overall ATS score badge, categorized extracted skills (Frontend, Backend, Database), detected projects, and AI-suggested interview questions.
- **Step 4: Pre-Interview Lobby & Hardware Diagnostics:** Click 'Start Interview'. Before the live round starts, the candidate enters the Pre-Interview Lobby. Here, WebRTC streams the live webcam feed and an audio visualizer monitors microphone levels. The candidate selects target role (e.g. Full Stack Developer), difficulty (Easy/Medium/Hard), and interview type.
- **Step 5: Live Voice-to-Voice AI Interview Cockpit:** Enter the live round. Explain the three core events: (1) Fullscreen lock triggers automatically; (2) The AI speaks Question #1 aloud using Web Speech Synthesis while displaying the question text; (3) The candidate clicks 'Speak Answer' or speaks naturally, and the Web Speech API transcribes speech in real time into the answer box. Click 'Submit Answer' to send the response to the backend for instant Gemini evaluation and adaptive next-question generation.
- **Step 6: Proctoring Guards in Action (Anti-Cheating Demo):** Explain how anti-cheating works in the UI: if the user switches tabs or presses Alt+Tab, the document visibility API triggers a warning modal ('Warning 1 of 3: Tab switch detected'). On the 3rd violation, the test auto-submits. If the candidate stops speaking or interacting for 90 seconds, a 30-second warning countdown pops up.
- **Step 7: Competitive Coding Arena (390 Problems):** Switch to Coding Arena. Point out the left sidebar containing 390 authentic LeetCode problems filtered by 20+ patterns (Two Pointer, Sliding Window, DP, etc.). Show the Monaco Code Editor (VS Code engine). Candidate selects Python 3 or JavaScript. Click 'Run Code' to test against sample test cases or 'Submit' to validate against all test cases. The output terminal displays runtime, memory, and stdout.
- **Step 8: Final Report & Scorecard Generation:** Upon finishing the interview, the candidate is routed to the Report Page. Highlight the Recharts radar chart displaying Technical, Communication, Relevance, and Completeness scores. Point out the Filler Word counter (e.g. 'um', 'like', 'actually') and Mistake Timeline. Click 'Download PDF Report' to show dynamic ReportLab PDF export.

## 4. Codebase Explanation: Folder by Folder & File by File

The project follows a clean, decoupled **Client-Server Architecture**. The backend is built with FastAPI (Python) and SQLAlchemy ORM; the frontend is built with React 18, Vite, and TypeScript.

| Directory / File Path                | Layer           | Purpose & Key Logic Contained                                                                                                                                                                                  |
|--------------------------------------|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| backend/app/main.py                  | Backend Entry   | FastAPI application instance, CORS middleware, API prefix rewrite middleware, router inclusions, and lifespan handler running DB table creation and seeding in a daemon thread so Render binds port instantly. |
| backend/app/database.py              | Database        | Creates SQLAlchemy engine with Neon PostgreSQL URL, connection pooling, and Base declarative class for all ORM models.                                                                                         |
| backend/app/models.py                | ORM Models      | Defines 13 relational tables: User, Interview, Question, Answer, InterviewReport, CodingProblem, CodingSubmission, PracticeQuestion, PracticeSubmission, Resume, ResumeAnalysis, ProctoringLog.                |
| backend/app/schemas.py               | Pydantic        | Pydantic v2 schemas validating request/response JSON payloads (e.g. UserCreate, Token, AnswerSubmit, CodeRunRequest, PracticeAttempt).                                                                         |
| backend/app/auth.py                  | Security        | Password hashing and verification using passlib bcrypt; JWT access token generation and cryptographic decoding.                                                                                                |
| backend/app/dependencies.py          | Auth Middleware | FastAPI dependency injection: get_db() yields database session with automatic closure; get_current_user() verifies JWT and returns current User object.                                                        |
| backend/app/routes/interviews.py     | API Route       | Handles interview lifecycle: /start creates session; /answer evaluates candidate answer & fetches adaptive next question; /finish aggregates scores.                                                           |
| backend/app/routes/coding.py         | API Route       | Endpoints to fetch 390 DSA problems, run code against sample test cases, and evaluate full submissions against hidden test cases.                                                                              |
| backend/app/routes/practice.py       | API Route       | Endpoints for categorized practice questions and real-time AI answer evaluation with rubrics.                                                                                                                  |
| backend/app/routes/resume.py         | API Route       | PDF resume upload, text extraction using PyPDF, and Gemini ATS scoring.                                                                                                                                        |
| backend/app/routes/reports.py        | API Route       | Endpoints to retrieve structured interview reports and stream downloadable PDF scorecards.                                                                                                                     |
| backend/app/services/gemini.py       | AI Service      | Core Gemini client with multi-model candidate pool (gemini-2.5-flash, gemini-3.5-flash-lite), JSON mode enforcement, resume parsing, question formulation, and rubric scoring.                                 |
| backend/app/services/coding.py       | Execution       | Sandboxed execution engine for Python 3 and JavaScript with timeout limits, memory isolation, and argument unpacking.                                                                                          |
| backend/app/services/pdf_report.py   | PDF Engine      | ReportLab script generating formatted candidate scorecards with custom canvas numbering.                                                                                                                       |
| frontend/src/App.tsx                 | Frontend Root   | React Router configuration with route guards, global Navbar, Footer, and AuthProvider wrapper.                                                                                                                 |
| frontend/src/context/AuthContext.tsx | State Context   | React context managing user login state, JWT token storage, user profile, and logout actions.                                                                                                                  |

|                                                   |            |                                                                                                                                                                                      |
|---------------------------------------------------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>frontend/src/pages/InterviewPage.tsx</code> | UI Page    | Live interview cockpit with Web Speech API integration (TTS reading + STT dictation), audio visualizer, fullscreen enforcement, tab-switch monitoring, and 90s inactivity countdown. |
| <code>frontend/src/pages/CodingPage.tsx</code>    | UI Page    | Interactive coding arena with Monaco Editor, problem search/filtering, Python/JS selector, test-case runner, and console output.                                                     |
| <code>frontend/src/services/api.ts</code>         | API Client | Axios instance configured with base backend URL and request interceptor that automatically attaches Authorization: Bearer <token> to every outgoing API request.                     |

## 5. Top Viva, Technical & Follow-Up Questions with Answers

Here are the exact questions examiners and technical interviewers ask about this architecture, with model answers:

### Q1: What is a 'Route' in Frontend vs. Backend?

**Frontend Route:** In React Router (e.g. `/interview` or `/coding`), a route is a client-side URL mapping that renders a specific React component without requesting a new HTML page from the server (Single-Page Application).

**Backend Route:** In FastAPI (e.g. `@router.post('/api/auth/login')`), a route is an HTTP endpoint mapped to an HTTP method (GET, POST, PUT, DELETE) and a Python handler function that receives requests, queries the database, and returns JSON.

### Q2: How does JWT (JSON Web Token) authentication work end-to-end?

- Candidate enters email & password on the frontend.
- Backend verifies password hash using `passlib.hash.bcrypt`. If valid, it encodes a JWT containing user ID, email, role, and expiration (exp) signed with a secret key using HS256 algorithm.
- Frontend receives the token and stores it in `localStorage`.
- On subsequent requests, Axios request interceptor attaches `Authorization: Bearer <token>` in headers.
- FastAPI's `get_current_user` dependency decodes and verifies the signature on protected endpoints without hitting the database for session state (stateless authentication).

### Q3: What is CORS and why did you configure it?

**CORS (Cross-Origin Resource Sharing)** is a browser security mechanism that restricts a web page from making requests to a different domain/port than the one serving the page. Because our React frontend runs on Vercel (or `localhost:5173`) and our FastAPI backend runs on Render (or `localhost:8000`), the browser blocks cross-origin requests by default. We added FastAPI's `CORSMiddleware` to explicitly allow origins, headers, and methods.

### Q4: Why did you choose FastAPI over Flask or Django?

FastAPI is built on **Starlette** (high-performance ASGI async web framework) and **Pydantic** (fast data validation). It supports native Python `async/await` for non-blocking I/O (critical for concurrent LLM and database calls), automatically generates interactive Swagger/OpenAPI documentation (`/docs`), and is significantly faster than Flask and Django for REST APIs.

### Q5: What is the difference between synchronous (`def`) and asynchronous (`async def`) endpoints in FastAPI?

In FastAPI, an `async def` endpoint runs directly on the central **asyncio event loop**, allowing the server to handle other incoming requests while waiting for I/O operations (like database queries or external API calls). A standard `def` endpoint runs inside an external threadpool executor so it doesn't block the main event loop.

### Q6: How does the AI Speech Recognition and Synthesis work without paid third-party voice APIs?

We leveraged the browser's native **Web Speech API**: `window.SpeechRecognition` for continuous client-side Speech-to-Text (STT) and `window.speechSynthesis` with `SpeechSynthesisUtterance` for natural Text-to-Speech (TTS). This eliminated third-party audio API latency, slashed operational cost to \$0, and kept speech processing on the client device.

### Q7: How did you optimize Google Gemini AI to prevent timeouts and rate limits?

We implemented a 4-pillar optimization strategy in `backend/app/services/gemini.py`:

- Model Prioritization:** Switched candidate models to `gemini-2.5-flash` (fastest, high quota) and `gemini-3.5-flash-lite`.
- JSON Schema Enforcement:** Configured `response_mime_type='application/json'` to guarantee valid JSON formatting without markdown formatting errors.
- Token Capping:** Set low temperature (0.2) to prevent verbose rambling and keep response latency under 4 seconds.
- Multi-Model Fallback:** If a candidate model times out or encounters a 429 quota error, the service automatically fails over to the next candidate model seamlessly.

### Q8: How does the Anti-Cheating / Proctoring engine work technically?

- Fullscreen Enforcement:** Calls `document.documentElement.requestFullscreen()` on interview start and listens for `fullscreenchange` events.
- Tab Switching Detection:** Listens to the browser's `visibilitychange` event and `document.hidden` property. If the tab loses focus, a counter increments. Two warnings are shown; the 3rd violation triggers automatic interview finalization.
- Inactivity Guard:** A timer resets on mouse, keyboard, or speech events. If 90 seconds pass with zero activity, a 30-second warning countdown modal is displayed before auto-submitting.

**Q9: How did you overcome Render free-tier sleep and startup timeouts?**

1. **Non-Blocking Startup:** Database table creation (`Base.metadata.create_all`) and seeding 390 problems to Neon PostgreSQL runs in a daemon background thread (`_init_db_background`). This allows Uvicorn to bind the port immediately ( $< 2s$ ), preventing Render's 30s port binding timeout.
2. **24/7 Keep-Alive Monitor:** A Google Apps Script time-driven trigger pings the root endpoint (`/`) with an HTTP GET request every 5 minutes, preventing Render's 15-minute inactivity spin-down and ensuring zero cold-start delay for users.

**Q10: What is an ORM and why use SQLAlchemy 2.0 with Neon PostgreSQL?**

An **Object-Relational Mapper (ORM)** maps database tables to Python classes and rows to Python objects. SQLAlchemy 2.0 provides type-safe queries, prevents SQL injection through automatic parameter binding, handles database migrations, and manages connection pooling. Neon PostgreSQL provides cloud-native, serverless PostgreSQL with automated SSL encryption and high availability.

## 6. Summary of Architectural Achievements

- **Full-Stack Cohesion:** Seamless integration between React 18, TypeScript, FastAPI, SQLAlchemy, and Neon PostgreSQL.
- **390 Authentic Problems:** Complete LeetCode/GFG problem inventory with genuine inputs, outputs, starter code, and test runners.
- **Sub-Second Responsiveness:** Optimized Gemini 2.5 Flash pipeline delivering answers and evaluations within 4 seconds.
- **Production Live:** Fully deployed and continuously monitored on Render, Vercel, and Neon with zero-downtime architecture.